

SKALA

AI 서비스를 위한 SW 기초 Full-stack Engineering

스마트 데이터 이해 및 활용 (Trend)

2026.7

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

AI 서비스를 위한 SW 기초 Full-stack Engineering - 스마트 데이터 이해 및 활용

0. 과정 시작 전에...

01

OpenAI의 PostgreSQL 확장 사례

8억 명 사용자를 단일 프라이머리로 지탱한 전략

02

벡터 DB와 기존 패러다임의 결합

그래프 DB에 벡터 검색을 통합한 실제 제품 사례

03

벡터 DB 성능의 반전

코어를 늘려도 느려지는 HPC 환경의 역설

04

강의, 토론에 활용할 시사점

트레이드오프 사고와 비판적 성능 평가

OpenAI, PostgreSQL로 8억 명 사용자 지원하기

8억 명+

ChatGPT / API
월간 사용자

~50대

전 세계 분산
읽기 복제본

10배+

지난 1년간
DB 규모 성장

1대

쓰기 담당
단일 프라이머리

핵심 인프라 구조

- 단일 프라이머리(Azure PostgreSQL Flexible Server)가 모든 쓰기를 처리
- 초당 수백만 건 쿼리(QPS) 처리, p99 지연시간은 낮은 두 자릿수 밀리초 유지
- 읽기 트래픽은 지역별로 분산된 복제본으로 오프로드

"샤딩된 분산 DB로 전면 전환하지 않고, 검증된 기술을 한계까지 최적화"

OpenAI의 문제와 대응 전략

문제	대응 전략	관련 개념
단일 프라이머리는 쓰기 확장 불가	샤딩 가능한 쓰기 워크로드를 Azure CosmosDB 로 이전 (polyglot persistence)	분산 시스템, NoSQL
MVCC 로 인한 쓰기 증폭	UPDATE 시 로우 복사 → 오래된 버전 정리(vacuum) 오버헤드 발생	MVCC , 트랜잭션
스키마 변경이 테이블 전체 재작성 유발	경량 변경만 허용, 5초 타임 아웃 강제 적용, 신규 테이블 원칙적 금지	DDL , 운영 정책
연결 폭주 (Connection Storm)	PgBouncer 로 커넥션 Pooling , Kubernetes 배포	커넥션 관리
복제본 증가 시 프라이머리 부하 증가	Azure 와 협업해서 계단식 복제 (Cascading replication) 개발 중	복제 아키텍처

OpenAI의 문제와 대응 전략(상세)

문제	왜 문제인가?	해결책
쓰기 확장 불가	프라이머리 1대가 모든 INSERT/UPDATE를 처리	확장 가능한(shardable) 워크로드는 Azure CosmosDB(NoSQL)로 → "폴리글랏 퍼시스턴스(polyglot persistence)" 즉 "하나의 DB로 다 해결"이 아니라 "워크로드 성격에 맞게 여러 DB를 섞어 쓴다"
MVCC 쓰기 증폭	PostgreSQL은 UPDATE할 때 기존 로우를 지우지 않고 새 버전을 추가로 생성 (Multi-Version Concurrency Control) - 쓰기가 많으면 이 오래된 버전들이 쌓여서 디스크 공간과 vacuum(청소) 부하가 커짐	- 쓰기 자체를 줄이는 애플리케이션 레벨 최적화 + vacuum 스케줄 관리

OpenAI의 문제와 대응 전략(상세)

문제	왜 문제인가?	해결책
스키마 변경 위험	- 컬럼 타입 변경 같은 스키마 변경 시 테이블 전체를 다시 쓰는(rewrite) 작업을 유발 가능 - 큰 테이블이면 몇 시간씩 잠금(lock) 발생	- 가벼운 변경(컬럼 추가/삭제 등)만 허용 - 모든 스키마 변경에 5초 타임아웃 강제 (5초 안에 안 끝나면 자동 실패로 Prod. 영향 없게)
연결 폭주	- Azure PostgreSQL은 최대 연결 수가 5,000개로 제한 - 트래픽 급증 시 연결 소진되면 서비스 전체가 죽음	PgBouncer(커넥션 풀러)로 연결을 재사용 ("다 같이 차 한 대씩 타지 말고 카풀하자")
복제본 부하	프라이머리가 WAL(Write-Ahead Log, 트랜잭션 기록)을 복제본 50대 전부에 직접 전송 → 복제본 증가 시 부담 증가	- Azure와 함께 계단식 복제(cascading replication) 개발 중 - 중간 복제본이 다른 복제본에게 릴레이해주는 방식

벡터 검색, 그래프 DB의 표준 기능이 되다

무엇을 했나

- TigerGraph(MPP 기반 네이티브 그래프 DB)에 벡터 검색 기능을 통합
- 정점(vertex) 속성에 임베딩 타입 추가 + 전용 MPP 인덱스 프레임워크 설계
- 그래프 쿼리 언어 GSQL을 확장해 벡터 검색과 그래프 쿼리를 조합
- Neo4j, Amazon Neptune, 전문 벡터 DB Milvus 대비 우수한 성능을 실험으로 입증
- 2024년 12월, TigerGraph v4.2에 실제로 제품 반영됨

의미

벡터 검색이 더 이상
“별도의 특수 DB”가 아닌
관계형, 그래프 DB의 표준 기능
으로 흡수되는 흐름
(실제 **Production** 사례)

[참고] MPP(Massively Parallel Processing)

단일 DB 인덱스(예: B-Tree)는 데이터가 수억~수십억 건 이상 시
MPP 분산 환경 지원 불가

[참고] MPP(Massively Parallel Processing)

MPP 란?

- 여러 개의 독립된 컴퓨팅 노드
(프로세서+메모리+디스크)가
데이터 처리 작업을 나누어 동시 실행 구조
- **Shared Nothing**
 - 각 노드가 자신만의 메모리와 디스크를 독자적으로 사용하며 서로 자원을 공유하지 않음
- 데이터가 커지면 서버(노드)를 옆으로 추가하기만 하면 되므로 확장성(Scale-out)이 강점
- 대규모 데이터 웨어하우스(DW) 및 빅데이터 분석 플랫폼
(Amazon Redshift, Snowflake, Google BigQuery, Teradata 등)

분산 데이터 탐색 극대화
: 각 분산 노드에 저장된 블록
/ 파티션 단위 데이터에서
정확한 위치를 빠르게 찾아내어
불필요한 전체 데이터 스캔(**Full Scan**)을 최소화

병렬 조회 성능 향상
: 데이터 검색 시 여러 노드가
인덱스를 기반으로 병렬 처리를
수행하므로 초고속 조회가 가능

벡터 검색, 그래프 DB의 표준 기능이 되다

【 배경 】

- RAG(검색 증강 생성)를 할 때
보통 "벡터 DB에서 유사 문서 찾기"와
"그래프 DB에서 관계 탐색하기"를 별도
시스템으로 분리
- TigerVector는 이 둘을 하나의 그래프
DB(TigerGraph) 안에서 처리할 수 있게
만든 시스템

기술적으로 한 일

그래프의 정점(vertex, 노드)에
"임베딩(embedding)"이라는 새 속성 타입을 추가
(그래프의 각 노드가 벡터값도 가질 수 있게)

이 벡터에 대해 빠르게 검색할 수 있는 전용 인덱스를
만들고, 이걸 MPP(Massively Parallel Processing,
대규모 병렬 처리) 그래프 엔진과 통합

쿼리 언어 GSQL을 확장해서
"벡터로 유사한 것 찾고 → 그 결과에서 그래프 관계
따라가기" 같은 복합 쿼리를 한 번에 작성 가능하게 함

벡터 검색, 그래프 DB의 표준 기능이 되다

기술적으로 한 일

【 배경 】

- RAG(검색 증강 생성)를 할 때

보통
"그래프
시스템"

- TigerGraph DB(1)
만든

SIGMOD 2025(데이터베이스 분야 최고 학회 중 하나)의 산업 트랙에 채택

실제로 2024년 12월 TigerGraph 정식 제품(v4.2)에 반영

즉 학술적 아이디어에 그치지 않고 실제 상용 제품이 됐다는 게 핵심 포인트

그래프의 정점(vertex, 노드)에
"임베딩(embedding)"이라는 새 속성 타입을 추가
(그래프의 각 노드가 벡터값도 가질 수 있게)

스를
ing,

쿼리 언어 GSQL을 확장해서
"벡터로 유사한 것 찾고 → 그 결과에서 그래프 관계
따라가기" 같은 복합 쿼리를 한 번에 작성 가능하게 함

LLM × 벡터 DB 결합 연구 동향

When Large Language Models Meet Vector Databases: A Survey

arXiv:2402.01763

LLM과 벡터DB(VecDB) 결합의 이론적 근거, 응용 프로토타입 분류, 연구/엔지니어링 과제를 체계적으로 정리한 서베이

→ LLM의 환각(hallucination)을 줄이기 위해 벡터DB에서 검색한 사실을 근거로 답하게 하는 구조 등을 유형화

LLM × 벡터 DB 결합 연구 동향

A Comprehensive Survey on Vector Database: Storage and Retrieval Technique, Challenge

arXiv:2310.11703 (v4, 2026.3 개정)

고차원 벡터 데이터가 기존 DBMS의 한계를 넘어서며 벡터DB가 LLM과 긴밀히 결합되는 과정을 스토리지·검색 두 축에서 추적

→ 벡터DB 자체의 기술(저장 방식, 검색 알고리즘)을 스토리지 축과 검색 축으로 나눠 정리
2026년 3월까지 계속 개정되고 있는 "살아있는" 논문

반증 사례: “코어를 늘려도 더 빨라지지 않는다”

실험 설계

Qdrant · Milvus · Weaviate 3개 벡터DB를 실제 슈퍼컴퓨터 2대(Polaris, Aurora)에서 최대 64개 노드, 256개 분산 워커까지 확장하여 평가 (자체 과학 데이터셋 Pes2o-VE: 임베딩 8,800만 개, 843.56GB 포함)

최대 30.67%

코어 추가 투입 시
쿼리 처리량 감소

16배 → 5.46배

워커 16→256배 확장 시
실제 성능 개선 폭

2.8배

Milvus 쓰기 증폭
(95GB → 266GB)

**“클라우드 환경에 맞춰 설계된 벡터DB 아키텍처를 그대로 HPC로 옮기면,
하드웨어를 늘려도 성능이 오히려 나빠질 수 있다”**

→ 인덱싱·성능 튜닝은 워크로드와 실행 환경에 강하게 종속적이라는 원칙을 뒷받침하는 최신 실증 사례

반증 사례: “코어를 늘려도 더 빨라지지 않는다”

【 실험 】

- Argonne 국립연구소 등이 실제 슈퍼컴퓨터(Polaris, Aurora) 2대에서 Qdrant, Milvus, Weaviate 세 벡터DB를 최대 256개 워커로 확장하며 테스트

3가지 결과

코어 더 투입 시 오히려 처리량이 최대 **30.67%** 감소

- 일반적으로 "자원을 더 주면 더 빨라진다"고 생각
- 벡터DB의 내부 동시성 제어 방식이 클라우드 환경 기준으로 설계되어 HPC의 고성능 네트워크, 스토리지 환경과 맞지 않아 오히려 병목 발생

반증 사례: “코어를 늘려도 더 빨라지지 않는다”

【 실험 】

- Argonne 국립연구소 등이 실제 슈퍼컴퓨터(Polaris, Aurora) 2대에서 Qdrant, Milvus, Weaviate 세 벡터DB를 최대 256개 워커로 확장하며 테스트

3가지 결과

워커 16배(16→256개) 증설에도 성능은 5.46배만 개선

- 이상적으로 16배 늘리면 16배 빨라져야 하나(선형 확장)
- 실제로는 그 3분의 1 수준밖에 못 미칩니다.

**Milvus는 95GB 데이터가 266GB로 커짐
(2.8배 쓰기 증폭)**

- 페타바이트급 과학 데이터를 다루는 환경 시 저장 공간 낭비가 치명적

반증 사례: “코어를 늘려도 더 빨라지지 않는다”

3가지 결과

【 실험 】

워커 16배(16→256개) 증설에도 성능은 5.46배만 개선

최신 기술 = 무조건 좋다"가 아니라
"설계된 환경(클라우드)과 실제 배치 환경(HPC)이 다르면 성능이 역행할 수 있다"는
실측 데이터로 보여주는 사례

(2.8배 쓰기 증속)

- 페타바이트급 과학 데이터를 다루는 환경 시 저장 공간 낭비가 치명적



수고하셨습니다.

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.